



Debouncing

Learn In 5 minutes

Manzer Ansari

 @bug_and_debug

What is Debouncing?

Debouncing limits how often a function is called.

It **waits for the user to stop typing/moving/etc** before firing the action.

Think:

Don't run the function until the user finishes what they're doing.



Manzer Ansari

 @bug_and_debug

Real-World Use Cases

-  Search bar suggestions
-  Live filtering
-  Window resize
-  Auto-save as you type
-  Reducing API calls in forms

Manzer Ansari

 @bug_and_debug

Why is Debouncing Important?

Without debounce:

- 🔄 onChange or scroll triggers too many events
- 🔥 Can overload your API or UI

With debounce:

- ✅ Only one API call after the last input
- ✅ Smoother performance
- ✅ Saves bandwidth

Manzer Ansari

📷 @bug_and_debug

Debounce in React (Basic)

```
const handleChange = debounce((value) => {  
  fetchData(value);  
}, 500);  
  
<input onChange={(e) => handleChange(e.target.value)} />
```

✓ No API call until user pauses typing

Manzer Ansari

 @bug_and_debug

React + useEffect + debounce

```
useEffect(() => {  
  const timer = setTimeout(() => {  
    fetchData(search);  
  }, 500);  
  
  return () => clearTimeout(timer); // cleanup  
}, [search]);
```

- ✓ No external library needed
- ✓ Works well for simple use cases

Manzer Ansari

 @bug_and_debug

Summary

Debouncing is **not optional** when you're building:

- Search bars
- Real-time input
- Resizing
- Scroll-based actions
- ✦ It improves performance
- ✦ Reduces load
- ✦ Makes UX feel snappy

Manzer Ansari

 @bug_and_debug

Enjoyed this post?

Feel free to **save and share** it with others!

Connect with me on

 Manzer Ansari

 @bug_and_debug